

Simulace paprskové optiky s grafickým rozhraním

Dokumentace k maturitní práci



Daniel Matouš
Gymnázium, Praha 4, Písnická 760
22. dubna 2026

Abstrakt

Tato dokumentace popisuje grafické rozhraní pro program na simulaci paprskové optiky ve 2D prostoru. Cílem bylo vytvořit desktopovou aplikaci, jejíž GUI má všechny náležitosti očekávané od moderního rozhraní. Dal jsem si za cíl nepoužít jiné knihovny než ty, které jsou součástí základního Pythonu 3.14; všechny systémy projektu jsem se chtěl naučit vymyslet a implementovat sám.

Pro projekt jsem vytvořil základ optické simulace, která v aktuálním stavu podporuje bodové zdroje libovolného počtu paprsků, odražení od zrcadel a překážky blokující paprsky. Podporuje přidávání dalších prvků odvozením od obecných tříd.

Zásadním bodem cíle byl vlastní systém vybírání a přesouvání simulačních prvků pomocí myši metodou "Drag n' drop".

GUI bylo opatřeno i méně častými widgety a ve svém aktuálním stavu vyladěno, ale není dost vybavené funkcemi, aby dovolilo uživateli využít všech možností simulace. Simulace sice poskytuje funkční základ, na kterém se dá expandovat, ale postrádá schopnosti, které jsem chtěl přidat.

V průběhu tvorby jsem totiž zjistil, že bude komplikovaným výkonem zprovoznit i jen úplný základ. To se mi nakonec povedlo a v této dokumentaci vás provedu jeho tvorbou, fungováním a dalším využitím.

Obsah

1	Úvod	1
2	Programátorská dokumentace	3
2.1	Použité technologie a nástroje	3
2.2	Využívané programovací koncepty	5
2.3	Drag 'n drop	6
2.4	GUI	7
2.5	Simulace	8
2.6	Debugovací funkce	11
2.7	Známé buggy	11
2.8	Architektura souborů projektu	12
2.9	Formátování kódu	13
3	Uživatelská dokumentace	15
3.1	Instalace a spuštění	15
3.2	Popis GUI	16
3.3	Použití	18
3.4	Režimy výběru myši	19
3.5	Řešení problémů	19
4	Závěr	20

1 Úvod

Jakmile jsem zjistil, že předmětem závěrečného ročníku bude tvorba maturitního projektu, tak jsem věděl, že chci tvořit nějaké grafické rozhraní. Tím jsem si byl od začátku jistý nejen proto, že to je oblast, která mě zajímá a chci se v ní zlepšovat, ale také proto, že v budoucnu bych chtěl pracovat právě na vývoji nástrojů s GUI. Dále mi bylo jasné, že budu chtít tvořit čistě softwarový projekt bez hardwarové součásti, protože s prací s hardwarem nemám žádné dobré zkušenosti.

Výběr platformy pro mě byl také jasný. Mám zkušenosti s Pythonem a jeho knihovnou Tkinter, které bych si rád prohloubil.

Jako obsah své práce jsem si vymyslel vykreslování paprskové optiky, protože to je záležitost, která se bez dobrého grafického rozhraní neobejde, a proto jsem počítal s tím, že nechá práci na GUI vyniknout. Při sepisování požadavků na svou simulaci jsem počítal s využitím jako program, který by studenti mohli využít při studiu optiky pro prohloubení intuice nebo profesori pro vytváření obrázků optických soustav bez nutnosti používat online nebo placené nástroje. V plánu jsem tedy měl plno simulačních prvků, které by zajistily, že pomocí nich půjde vytvořit skoro cokoliv, co by zamýšlený uživatel mohl chtít.

Udělat tak všestranný program, aby se stal pro někoho první volbou pro práci s optikou, se mi upřímně nepovedlo.

Existují již nástroje, které dělají to, co můj, a k tomu ještě mnohem víc. Na jeden takový nástroj, PhyDemo Ray Optics Simulator, mě upozornil můj profesor hned, jak jsem tento nápad na projekt navrhl. Mým cílem zde ale není vyvinout program schopný konkurovat schopnostmi projektům od mnohem zkušenějších vývojářů (nebo týmů vývojářů), než jsem já, nýbrž ukázat na funkčním příkladu, co umím vytvořit pomocí své oblíbené GUI knihovny.

Cílem tohoto tématu bylo naučit se co nejvíc o tvorbě uživatelského rozhraní a konkrétně o tvorbě pomocí Tkinteru. V tomto ohledu jsem uspěl, protože kromě základních funkcí úspěšně využívám pokročilejší schopnosti knihovny, včetně používání značek (tagů) pro identifikaci objektů na plochách a dále včetně událostmi řízeného programování a využití rozšiřujícího modulu Tkinter.ttk.

Naučil jsem se hodně jak o principech tvorby grafického rozhraní a mnoha často používaných konceptech (práce s mřížkou, resizování, drag 'n drop, událostmi řízené programování), tak o Tkinteru samotném (tvorba menu lišty, systém událostí v Tkinteru, Canvas tagy). Dokážu nyní v této knihovně efektivně pracovat a chápu její součásti. Důvěrně jsem se seznámil i s widgety z jejího rozšiřujícího modulu Tkinter.ttk (ttk = Themed TKinter).

V této dokumentaci najdete informace k fungování projektu a k tomu, jak jej používat. Nejdříve naleznete programátorskou dokumentaci, pak následuje uživatelská dokumentace. V programátorské části je vysvětlena struktura mého projektu a jsou tam popsány nejdůležitější jeho součásti. Popsáno je nejprve také prostředí, ve kterém jsem pracoval a rovněž použité knihovny. Uživatelská dokumentace se sestává z návodu na instalaci a spuštění programu a z návodu k použití.

2 Programátorská dokumentace

V programátorské dokumentaci vás seznámím nejdříve s použitými vývojovými nástroji, pak základními principy, na kterých můj projekt funguje a potom vše rozeberu více do hloubky v částech postupně o stavbě grafického rozhraní, souborové struktuře, matematické simulaci a nakonec popíšu, jak se v kódu projektu orientovat a přiblížím vše potřebné pro ty, kdo by v něm chtěli dělat změny. Relevantní příklady kódu, na kterých chci něco ukázat, budou provázet celou tuto kapitolu.

2.1 Použité technologie a nástroje

- Programovací jazyk: Python. S Pythonem mám předchozí zkušenosti a hodí se na tento typ projektu proto, že má dobré možnosti pro práci s OOP a je připravený na využití knihovny Tkinter, kterou chci používat.
- Knihovny:
 - Tkinter: Defaultní knihovna Pythonu, název je zkratka z "Tk Interface". Knihovna Tkinter byla vytvořena v roce 1991 a rozhraní v ní vytvořená vypadají dnes krajně zastarale. Mě se tento vzhled líbí, proto toto nepovažuji za negativum a pokusil jsem se tohoto stylu držet. V Tkinteru lze tvořit i moderně vypadající okna, a to obzvlášť díky knihovně "ttk", která dovoluje vytvářet vlastní styly.
 - Modul Tkinter.ttk: Modul, který je součástí knihovny Tkinter. Je to zkratka z "Themed Tk" a dovoluje používat několik nových grafických prvků a definovat a používat vlastní styly. Já z Tkinter.ttk používám prvky PanedWindow, Separator, LabelFrame, Notebook a Sizegrip, které mnou vytvořenému grafickému rozhraní dávají dnes již časté moderní vymoženosti.
 - Modul Tkinter.filedialog: Modul, součástí Tkinteru, který poskytuje předpřipravené funkce pro vyvolání okna na výběr souborů

pro načtení nebo uložení. Okna vypadají tak, jak je uživatel zvyklý ze všech ostatních programů, které dovolují otevírání a tvorbu souborů a jejich vzhled se přizpůsobuje vzhledu operačního systému.

- Math: Standardní knihovna Pythonu, která do mého programu dodává matematické funkce potřebné při výpočtech v analytické geometrii. Zároveň zachránce při geometrických výpočtech, zároveň zdroj jednoho z nejabsurdnějších bugů mého projektu, popsaného níže. Z Math používám funkce `radians`, `sin`, `cos`, `sqrt`, `atan2`, `degrees`
- Random: Standardní knihovna Pythonu, kterou využívám pouze pro generaci náhodných čísel do jmen objektů pro jejich odlišení.
- Json: Knihovna, která zprostředkovává konverzi Python objektů do textové reprezentace ve formátu Json a také konverzi zpět z Json formátu na Python objekty. Používám tyto funkce pro uložení dat do souboru a načtení ze souboru.
- IDE: Visual Studio Code s rozšířením pro Python. Již s ním mám zkušenosti a přes přechodné problémy se zprovozněním Python debuggeru, které jsem vyřešil během začátku projektu, funguje bezproblémově.
- Verzovací systém: Git se vzdáleným repozitářem uloženým na Githubu. Jakožto budoucí student informatiky potřebuji, aby mi práce s Gitem šla od ruky. Tento projekt mi poskytl výbornou možnost se s Gitem, se kterým jsem neměl předchozí zkušenosti, naučit.
- Dokumentace: Vytvořena v LaTeXu za pomoci stránek Overleaf. Používám knihovny Graphics, Listings a Xcolor a další.

Během žádné části projektu nebyla využita AI.

```
1     vector_line = (line_x2 - line_x1, line_y2 - line_y1)
2     ...
3     return_dict["line_angle"] = degrees( atan2(
        vector_line[1], vector_line[0]) )
```

Listing 2.1: Využití `math.atan2()` v geometrickém výpočtu

Funkce `math.atan2(y, x)` mi způsobila asi týdenní zaseknutí, při kterém jsem přezkoumával každou část svých funkcí na geometrické výpočty, abych našel záludnou chybu. Požaduje totiž jako argumenty souřadnice vektoru a to

v pořadí y , x , čehož jsem si ve spěchu, abych konečně implementoval odrazy, nevšiml. Argumenty jsem podával v pořadí x , y a moje kolize pak vracely úhly nesprávné, ale dostatečně smysluplné na to, aby člověk začal podezřívat svoje vzorce a svoje chápání analytické geometrie, nikoliv nenápadné dosazení do funkce.

2.2 Využívané programovací koncepty

Všechny části projektu ve vysoké míře využívají OOP. Většina souborů projektu poskytuje třídy, se kterými pak `main.py` pracuje. Vstup je zajišťován za pomoci Tkinteru prostřednictvím funkcí `Widget.bind()` a vestavěného systému událostí (eventů). Využito je tedy událostmi řízené programování, zatímco hlavní proces je po inicializaci GUI ponechán v nekonečném cyklu `Tk.mainloop()`.

Kromě událostí (jako třeba eventů myši `<Motion>`, `<1>` a `<B1-Motion>` a virtuálních eventů `<<ComboboxSelected>>` a `<<NotebookTabChanged>>`) používám metodu `trace` náležící Tkinterovským třídám `StringVar` a `IntVar` (a `DoubleVar` a `BooleanVar`, ty ale nepoužívám) a dovolují nastavit callback funkci, která má být zavolána v momentě, kdy je do té proměnné vepsána nová hodnota. Tohle se mi velmi hodilo u dynamicky generovaného Spinbox widgetu v editovacích panelech - má totiž tři způsoby, jak v něm hodnota může být změněna, a přiřazení funkce pomocí `StringVar.trace` si se všemi třemi elegantně poradí univerzálním způsobem.

```
1      # Width entry field
2      self.var_width = tk.StringVar()
3      self.var_width.set(self.x1 - self.x0) # Set to
4      current width
5      self.var_width.trace("w", self.edit_size) # Takes
6      care of all three types of setting
7
8      self.spb_width = tk.Spinbox(master=frm, textvariable=
9          self.var_width, from_=1, to=100000)
10     self.spb_width.grid(row=1, column=1)
```

Listing 2.2: Přiřazení metody `edit_size` pomocí Tkinter `trace`

2.3 Drag 'n drop

Zásadním bodem cíle byl vlastní systém vybírání a přesouvání simulačních prvků pomocí myši metodou "Drag n' drop", které dovoluje rychle i přesně přemísťovat prvky a je nepostradatelnou součástí každého moderního GUI ovládajícího program, jako je můj.

Drag n' drop je vytvořen za pomoci systému eventů přítomném v Tkinteru. Když uživatel stiskne tlačítko, je zavolána funkce `get_mouse_selected`, která dostane jako argument událost, ze které získá souřadnice, na kterých událost vznikla, tedy souřadnice kliknutí. Zeptá se Tkinter Canvas widgetu, jaké objekty se na tomto místě nacházejí. Jako odpověď dostane interní ID objektů na Canvasu. Pro každé toto ID se poté zeptá canvasu, jaký je typ toho objektu. Všechny objekty, které jsou čáry, jsou ignorovány. Tímto získá seznam překážek, které uživatel právě chytil. Uloží je do seznamu. Pokud je aktuální režim vybírání "Vrchní překážka", vybere a uloží jen tu nejvíc vrchní z překážek, co najde pod kurzorem.

Zároveň vyvolá dynamické vygenerování editovacího panelu.

Pohnutí myši s podrženým levým tlačítkem vyvolá spouštění funkce `mouse_drag`, která tento seznam používá. Na všech těchto objektech zavolá jejich `move` metodu o příslušný počet pixelů. Aktualizuje také souřadnice zobrazené v editovacím panelu.

```

1 def get_mouse_selected(event: Event, screen: Screen) ->
  list[int]:
2     # Set variables based on selection mode
3     selection_mode = constants.selection_mode
4     if constants.debug_selection:
5         print(f"Mode: {selection_mode}")
6     match selection_mode:
7         case "SINGLE":
8             offset = constants.single_select_offset
9         case "MULTI":
10            offset = constants.multi_select_offset
11        case "LINES":
12            offset = constants.lines_select_offset
13        case _:
14            raise Exception("Selection mode does not
              match expected cases")
15
16    x0 = event.x - offset
17    x1 = event.x + offset
18    y0 = event.y - offset
19    y1 = event.y + offset
20    overlapping = screen.tk_canvas.find_overlapping(x0,
      y0, x1, y1) # From the documentation for tk.Canvas
      .find_overlapping(): "The items are returned in
      stacking order, with the lowest item first"
21
22    if len(overlapping) == 0: # If nothing selected,
      nothing selected
23        return []

```

Listing 2.3: Začátek funkce get_mouse_selected

2.4 GUI

Uživatelské rozhraní je rozděleno na tři panely. Levý panel je ponechaný prázdný, zde můžou být v budoucnu přidány dodatečné ovládací prvky, nebo doimplementován widget Treeview, který tento panel zatím vyplňuje a mohl by v budoucnu podávat přehled o hierarchii objektů na jednotlivých plochách. Funkce GUI jsou blíže popsány v uživatelské dokumentaci, zde se zaměřím na použité widgety.

Kopírovat sem kód inicializující GUI by byla škoda místa, proto raději výčet použitých widgetů:

- Z Tkinteru: Menu, Toplevel, Label, Frame, Labelframe, Button, Entry, Canvas, Spinbox
- Z Tkinter.ttk:
 - Notebook: Na přepínání mezi plochami systémem karet
 - Panedwindow: Rozdělení hlavní části okna na tři panely, jejichž hranice se dají vůči sobě posouvat
 - Separator: Dekorační detail - dělicí čáry v toolbaru a editovacím panelu
 - Sizegrip: Úchyty na chycení okna pro resizování. Hodně se mi líbí jejich vzhled, tak jsem použil rovnou dva
 - Combobox: Výběry několika věcí: Režim drag 'n drop výběru, typ a tvar přidávaného objektu, výběr plochy k odstranění

Během tvorby GUI jsem dbal na resizovatelnost okna (použitím několika Frame widgetů a nastavením resizování v grid manageru) a to jsem zohlednil i použitím widgetů Panedwindow a Sizegrip.

Chtěl jsem si vyzkoušet design vícero způsobů, jak něco konfigurovat, proto několik věcí používá ttk.Combobox, číselné vstupy pro šířku a výšku používají Spinbox, menu přidávání a odstraňování ploch používají Toplevel okna a textové vstupy u přidávání ploch používají widget Entry. Líbí se mi, že i přes moje malé množství předchozích zkušeností mi dovolil Tkinter zahrnout do projektu tak pestrý výběr rozhraní.

Jsem rád, že jsem si vyzkoušel i dynamické generování widgetů do editovacího panelu, protože to je něco, co jsem si už delší dobu chtěl vyzkoušet. Dekorační prvek Labelframe, který jsem použil na ohraničení editovacího panelu, se mi vzhledově líbí a těšil jsem se, až ho v nějakém projektu použiju.

2.5 Simulace

Vytvořil jsem systém pro ukládání a vykreslování segmentů paprsků na Tkinter canvas. Paprsek může být definován buď dvěma body, nebo jedním

bodem a jedním úhlem, což se hodí pro segmenty, u kterých není znám koncový bod, protože ještě neprošly kolizí nebo mají jít až do nekonečna a tedy být vykresleny až k okraji plochy.

Třídy podílející se na paprskové simulaci jsou `Source`, `Segment` a obecná třída `Interactor` a třídy od ní odvozené. Tyto třídy jsou všechny opatřeny typovými anotacemi jakožto i komentáři, proto je tato podkapitola neprobírá všechny do hloubky. Představím zde ale alespoň třídu `Segment` a pak `Source`.

Součástí simulace je také systém pro vyřizování kolizí a postupnou generaci nových segmentů, jak jde paprsek dál a podstupuje kolize, při kterých se generují nové jeho segmenty.

Níže jsou popsány dvě ze tříd tvořících kostru mého programu. Nacházejí se v `core_classes.py` a pokud chcete měnit něco v jádru programu, bude se vám hodit tento přehled.

```
1 class Segment:
2     def __init__(self, start_x: int, start_y: int, angle:
3         float, end_specified: bool = False, end_x: int =
4         None, end_y: int = None, visible: bool = True):
5         self.start_x: int = start_x
6         self.start_y: int = start_y
7         self.angle: float = angle
8         self._end_specified: bool = end_specified
9         self._end_x: int | None = end_x
10        self._end_y: int | None = end_y
11        self.visible: bool = visible
12        self.last_collided_interactor = None
13        self.last_collided_line = None
14    def __str__(self):
15        return f"Segment with X: {self.start_x} Y: {self.
16            start_y} Angle: {self.angle}"
17    def specify_end(self, x, y) -> None:
18        self._end_x = x
19        self._end_y = y
20        self._end_specified = True
```

Listing 2.4: třída `Segment`

`Segment` popisuje jeden segment paprsku. Je definován buď startovním bodem a úhlem, nebo startovním bodem a koncovým bodem. Moje renderovací

funkce se poradí s oběma způsoby definování. Často je segment vytvořen s informací o počátečním bodu a úhlu a později mu je určen koncový bod. Pro tento přechod lze použít funkci `specify_end(x, y)`, která nastaví správně proměnné. Je doporučeno používat tuto funkci, protože je kratší, než všechny tři proměnné nastavovat manuálně a také to zamezí případům, kde by člověk zapomněl nastavit proměnnou `_end_specified`. Proto jsou proměnné `end_x`, `end_y` a `end_specified` označeny jako soukromé atributy. Funkce `__str__()` je dostupná pro pohodlnější debugování.

```
1 class Source:
2     def __init__(self, x: int, y: int, angle: float) ->
      None:
3         self.x: int = x
4         self.y: int = y
5         self.angle: float = angle
6         self.generated_segments: list[Segment] = []
7     def __str__(self) -> str:
8         return f"Source with X: {self.x} Y: {self.y}
          Angle: {self.angle} Number of segments: {len(
            self.generated_segments)}"
9     def generate_segments(self, parent_screen: Screen) ->
      None:
10         ...
```

Listing 2.5: třída Source

Zdroj paprsku je definován jen svými souřadnicemi `x` a `y` a svým úhlem. Při spuštění funkce `Source.generate_segments` vygeneruje zdroj první paprsek s právě těmito souřadnicemi a úhlem a další segmenty k paprsku přidává podle výsledků kolizí. Aby zdroj paprsku mohl získat data o kolizích, potřebuje dostat při zavolání této funkce odkaz na instanci plochy, na které se nacházejí prvky, se kterými bude interagovat. Z instance plochy získá seznam simulačních prvků, se kterými postupně se všemi nechá pomoci funkcí z modulu `collisions.py` vypočítat kolize. Všechny kolize, které neuspěly, jsou odhozeny. Z kolizí úspěšných vybere tu nejbližší k začátku posledního svého segmentu, tedy tu, na kterou paprsek narazí jako první. Segmenty vygenerované touto kolizí přidá ke svému paprsku, tím, že je přidá do svého seznamu `generated_segments`. Opět je dostupná `__str__()` funkce, která vrací sešumírované podrobnosti o instanci.

2.6 Debugovací funkce

Projekt obsahuje debugovací funkce, které mi pomáhaly při hledání chyb a potvrzování, že vše funguje tak, jak má, a určitě pomůžou i všem ostatním, kdo by se chtěli v mém projektu pohrabat nebo do něj něco přidávat. Klasické debugovací nástroje jako VSCode s Python debuggerem samozřejmě stále jsou nejvíc důležitým nástrojem.

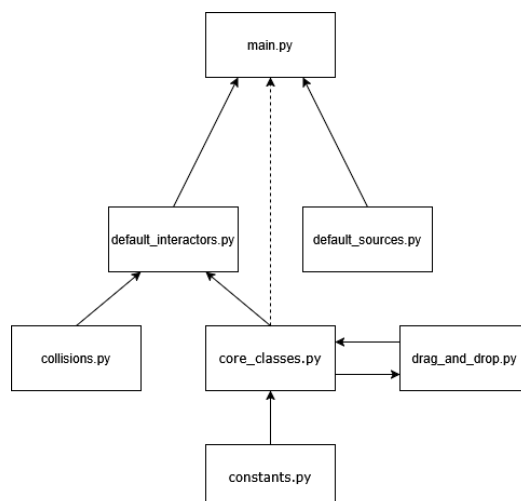
2.7 Známé bugy

Projekt není bezchybný. Když uživatel přepne na obrazovku, kterou předtím ještě neměl otevřenou - tedy jednu z předpřipravených obrazovek, kterou ještě neotevřel, nebo vlastní obrazovku načtenou ze souboru - tak mohou být paprsky vykresleny špatně, a to tím způsobem, že v místě změny směru jsou vykresleny jdoucí špatným směrem. Původ chyby by se dal podle mě hledat v části zodpovědné za kolize nebo za renderování paprsku. Když se mi nepovedlo najít zdroj chyby, vyřešil jsem problém přidáním tlačítka na aktualizaci obrazovky. Zároveň se program pokusí přerenderovat paprsky při přepnutí mezi plochami a díky tomu se často chybné vykreslení spraví i tím, když přepnete na jakoukoliv jinou plochu a pak zpátky na tu s chybným zobrazením.

V nějakých na první pohled naprosto náhodných případech selže výpočet kolizí s odrazy a paprsek se neodrazí. Problém patrně tkví ve funkcích na výpočty kolizí a odrazů, ale komplexita mého systému pro tyto výpočty mi nedovolila včas chybu opravit. Narazí-li uživatel na takovou chybu, stačí zrcadlem nebo zdrojem paprsku pohnout byť o jeden pixel.

2.8 Architektura souborů projektu

Hierarchie souborů kódu mého projektu je:



Obrázek 2.1: Graf dependencí

Šipka znamená, co importuje co. Součásti položené nahoře v hierarchii závisí na těch pod nimi a postupně na nich staví. Popis jednotlivých částí:

`constants.py`: Soubor konstant využívaných ostatními částmi, nahoře jsou konstanty měnitelné, kterými je možno přizpůsobit některé charakteristiky programu (vypnout načítání demonstračních ploch při spuštění, vypnout veškeré printování do konzole, změnit font dolní lišty, změnit barvu paprsků, změnit maximální povolený počet segmentů v paprsku, než sebekontrola programu automaticky ukončí paprsek...). Dole se nachází interně používané konstanty.

`core_classes.py`: Obsahuje hlavní třídy projektu - jeho základ.

`drag_and_drop.py`: Obsahuje kód pro výběr pomocí myši a přesun objektů. Stará se taky o vyvolání updatů pravého panelu.

`default_interactors.py`: Definuje předem vytvořené simulační prvky

`default_sources.py`: Definuje předem vytvořené generátory paprsků

main.py: Je určen ke spuštění programu, obsahuje kód pro GUI a vytváří demo plochy při otevření programu

2.9 Formátování kódu

Tato část se týká mých vlastních rozhodnutí o detailech toho, jak má být můj projekt formátován. Jsou to věci, o kterých píšu pro přiblížení toho, proč jsou některé věci napsané tak, jak jsou. Není to nic kriticky důležitého pro samotné fungování projektu, takže je to až tady jako poslední bod dokumentace pro programátory.

Soukromé atributy jsou označeny podtržítkem na začátku, třeba `_end_specified` ve třídě `Segment`. Python soukromé atributy neskrývá, proto můžete napsat kód, který je bude měnit přímo. To se ale nedoporučuje proto, že na práci s nimi jsou dostupné třídní metody, které jsou na jejich správné nastavení nebo přečtení přímo vytvořené.

Co nejvíc argumentům funkcí, atributům tříd a jiným proměnným jsem poskytl type anotace pro jednodušší práci s nimi a rychlejší hledání chyb.

```
1 class Source:
2     def __init__(self, x: int, y: int, angle: float,
3         editing_name: str="Zdroj", editing_type: str = "
4         Zdroj paprsku") -> None:
5         self.x: int = x
6         self.y: int = y
7         self.angle: float = angle
8         self.generated_segments: list[Segment] = []
9
10        # attributes specific to the editing panel
11        self._editing_name: str = f"{editing_name} #{id(
12            self)}"
13
14        self.editing_type: str = editing_type
15        self._editing_frame_generated: bool = False
16        self._editing_frame: None | tk.Frame = None
```

Listing 2.6: Konstruktor třídy `Source` opatřený typovými anotacemi argumentů i atributů

Co se týče uspořádání kódu v souborech, úplně nahoře jsou importy z jiných souborů mého projektu, pod nimi je volný řádek a pod tím jsou importy standardních Python knihoven.

```
1 import constants
2
3 import tkinter as tk
4 from tkinter import ttk # Just for one type annotation
   and a Combobox
5 from tkinter.filedialog import askopenfile, asksaveasfile
6 from math import tan, radians
7 from json import dumps, loads
```

Listing 2.7: Styl importů z core_classes.py

Názvy proměnných následují pravidla americké angličtiny, protože tu používají názvy v Pythonu a Tkinteru. Komentáře jsou napsané britskou angličtinou, protože tu používám já. Je to maličkost, kterou uvádím hlavně pro objasnění pro ty, kdo by se pozastavili nad tím, že používám jiný pravopis v názvu funkce a jiný v komentáři hned vedle. V kódu dbám na to, aby se pravopis shodoval s názvy proměnných v Tkinteru a v komentářích dbám, aby se shodovaly s mými poznámkami a mými shrnutími commitů v Git repozitáři.

Ohledně debugování

Jelikož člověk, který chce můj kód debugovat, má můj kód stejně otevřený v editoru, zapíná se debugování přímo přepsáním `True/False` v kódu. V `constants.py` je sekce určená nastavení, zda jsou debugovací funkce aktivní. Kromě hlavního nastavení, kterým lze zapnout a vypnout veškeré debugovací printování, je funkčnost rozdělena na několik druhů podle toho, jaký systém mého projektu chce člověk zkoumat. Může je zapínat postupně, aby konzole nebyla spamována nekonečnými print statementy, které si člověk nevyžádal. Nejpodstatnější nastavení:

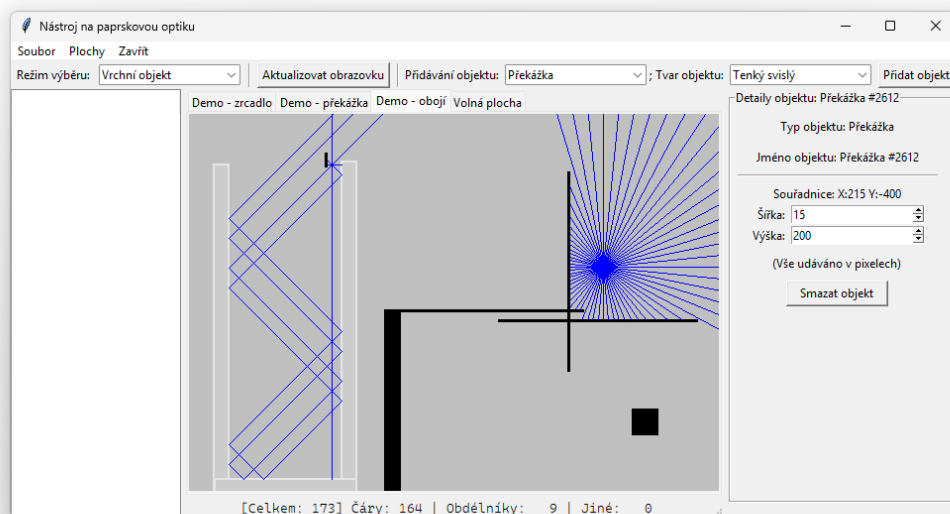
- `debug_any`: Hlavní nastavení. Pokud je vypnuto, vypne všechno debugování kromě vypsání zprávy při vypnutí.
- `debug_warnings`: Zapíná varování do konzole o věcech, které samy o sobě nejsou kritickou chybou, ale mohly by tropit problémy jinde, jako třeba počítání s úhly mimo rozmezí 0 až 360. Takovéto chyby se můj kód snaží nacházet a upozornit na ně. Všechna varování začínají "WARN: ". Pokud něco nefunguje a jsou při tom printována varování, zdroj těchto varování je první místo, kam by se člověk měl podívat, protože tam asi vězí chyba.

3 Uživatelská dokumentace

3.1 Instalace a spuštění

Pro používání stačí stáhnout složku obsahující soubory programu a spustit soubor `main.py`. Ten importuje všechny potřebné součásti a spustí okno programu. Po zavření okna se zavře zbytek programu. Projekt je závislý na aktuální verzi Pythonu, Python 3.14, a na knihovnách `Math` a `Tkinter`, obě jsou defaultní součástí Pythonu a jsou s každou jeho instalací defaultně předinstalované.

Stačí tedy stáhnout Python 3.14 a program spustit. Otevře se okno aplikace a načtou se předpřipravené demo plochy, na kterých si může člověk prohlédnout a vyzkoušet, co program umí.

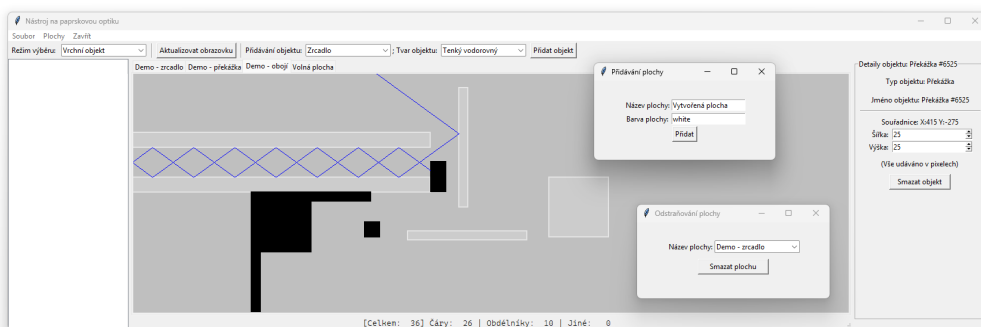


Obrázek 3.1: Grafické rozhraní s uživatelem vytvořenou simulací a spočtenými objekty

3.2 Popis GUI

Uživatelské rozhraní je rozděleno na tři panely. Prostřední panel je největší a nachází se na něm hlavní pracovní plocha programu. Při škálování aplikace také dostává prioritně nový prostor, zůstává tedy největším. V horní části naleznete lištu s několika záložkami. Myší a pak i šipkami na klávesnici lze mezi záložkami přepínat a tím určovat, která plocha je aktuálně zobrazena na hlavní ploše programu. Nahlédněte pod hlavní plochu a najdete lištu, na kterou se při každé změně na obrazovce vypíší aktuální údaje o tom, kolik je na dané ploše vykresleno čar, obdélníků a jiných útvarů. Celkový počet objektů je k přečtení nalevo. Použit je font typu monospace, aby se předešlo cukání při změně počtu cifer v číslech.

Panel napravo slouží k zobrazení detailů o vybrané položce. Pokaždé, když kliknete na nějaký objekt nebo jím myší přetáhnete, je vybrán a zobrazí se zde jeho typ a jeho název. Výběr zrušíte kliknutím na volné pozadí pracovní plochy. Jména objektů obsahují náhodná čísla pro odlišení od sebe. Zároveň se zde pod oddělovací čarou objeví při vybrání zrcadla nebo překážky editovací panel, ve kterém lze sledovat aktuální souřadnice objektu a také měnit výšku a šířku, což se ihned promítne na simulační ploše. Dole v editovacím



Obrázek 3.2: Referenční snímek GUI včetně vyskakovacích oken

panelu je tlačítko pro smazání objektu.

V horní části okna, nad hlavní částí, se nachází nástrojová lišta a nad ní tenká lišta s menu. Nástrojová lišta se sestává ze tří částí oddělených vertikálními čarami. V první lze nastavit režim výběrání myši pro systém drag 'n drop. V druhé se nachází tlačítko na aktualizování simulace a překreslení na obrazovku. Ve třetí části můžete ve výběru vybrat jeden z vícero druhů objektu, co přidat. Dostupné jsou zrcadlo, překážka a světelné zdroje o různém počtu paprsků. V druhém výběru v této části lze nastavit, jaký tvar bude mít přidané zrcadlo nebo překážka. Tlačítko hned vedle přidá na obrazovku objekt s požadovanými vlastnostmi. Pokud přidáváte světelný zdroj, pole na tvar objektu nemá žádný efekt. Tvar u zrcadel a překážek lze měnit podrobněji později pomocí editoru v pravé části aplikace.

Nad panely se nachází lišta menu se třemi tlačítky, sem lze v budoucnu přidávat další tlačítka. Jejich funkce jsou blíže vysvětleny v sekci níže.

3.3 Použití

Tlačítka v horní liště menu:

- Uložení/načtení souboru: Po otevření menu "Soubor" můžete uložit do souboru aktuálně vybranou plochu nebo načíst soubor tímto způsobem dříve uložený. Na příponě souboru nezáleží. Ukládá se název plochy, barva pozadí plochy, obsažené zdroje paprsku a obsažená zrcadla a překážky. Plocha z načteného souboru se objeví jako nová plocha v záložkách na vršku hlavní části okna.
- Přidání a smazání plochy: Po otevření menu "Plochy" stiskněte patřičné tlačítko a tím se otevře okno, kde můžete vybrat detaily a akci provést. U přidávání plochy lze nastavit její jméno, které se zobrazí nad ní jako titulek její karty, a také barvu pozadí. Barvu pozadí vepisujte jako hex kód ve stylu "#00FF00" nebo malými písmeny jako anglický název barvy. Slovní anglická pojmenování barev jsou podporována díky knihovně Tkinter, kterou můj program používá. U mazání plochy vyberete požadovanou plochu ve výběru ploch podle názvu. Vizte referenční snímek na předchozí stránce.
- Zavření: Použití tlačítka "Zavřít", křížku a zkratky Alt+F4 jsou ekvivalentní.

Hlavní plocha aplikace:

- Výběr objektu: Na objekt klikněte nebo jím myší přetáhněte.
- Přemístění objektu: Na objektu podržte levé tlačítko myši a někam ho přesuňte, poté pusťte.
- Editování objektu: Po kliknutí na objekt se objeví detaily o něm v editovacím panelu vpravo. U zrcadel a překážek zde můžete měnit jejich výšku a šířku. To lze provádět vepsáním čísla, tlačítka napravo od vepisovacího pole, nebo šipkami nahoru a dolů na klávesnici (po kliknutí někam do vepisovacího pole).
- Přepnutí na jinou plochu: Použijte lištu nad pracovní plochou. Pak lze použít i šipky doleva a doprava na klávesnici.
- Změna velikosti okna: Rukojeti na jednodušší uchopení okna se nachází v pravém dolním rohu okna a také pod pravým dolním rohem prostředního panelu.

- Změna velikosti panelů: Rukojeti na pohyb hranicemi panelů najdete na předělech mezi panely.

3.4 Režimy výběru myši

Vlastní implementace drag 'n drop systému je důležitá součást mého projektu. Implementoval jsem tři režimy, režim lze vybrat ve výběru vlevo nahoře v nástrojové liště. Defaultní je režim výběru jednoho objektu. Z objektů pod kurzorem vybere ten nejvíce v popředí. Režimy:

- Vrchní objekt: Z objektů přímo pod kurzorem vybere ten nejvíce v popředí
- Všechny pod kurzorem: Přetahování hýba všemi objekty pod kurzorem, k editování je vybrán ten nejvíce v popředí
- Zdroje světla: Vybere zdroje paprsků v těsné blízkosti kurzoru (není tedy potřeba trefit se úplně přesně, je zajištěno, že se kontroluje jistá oblast kolem kurzoru). Vybírají se zdroje paprsku, tedy je potřeba kliknout tam, kde paprsek začíná.

3.5 Řešení problémů

Pokud se simulace na obrazovce chová jinak, než by měla, stačí obsah obrazovky aktualizovat a problém by měl hned zmizet. To lze udělat kliknutím na tlačítko "Aktualizovat" nahoře na liště nástrojů, nebo přetažením jakýkoliv z překážek na obrazovce, protože to také způsobí celkovou aktualizaci obrazovky. Toto se může stát při přepnutí na novou obrazovku a tlačítko "Aktualizovat" toto řeší. Nejjednodušším řešením je ale pohnout jakýmkoliv prvkem na ploše, čímž se obsah plochy také aktualizuje a vykreslení se opět hned opraví.

4 Závěr

GUI se mi podařilo, jsem s ním spokojený. V aktuálním stavu funguje, jak by mělo a je zajištěno, že do budoucna na něj lze přidávat všechno, co může být potřeba.

Původně jsem zamýšlel, aby pracovní plochy byly scrollovatelné, ale to jsem v této verzi projektu nakonec neimplementoval. Zjistil jsem během práce, že by to bylo náročnější než jiné funkce, které pokládám za důležitější, zatímco přínos by nebyl tak vysoký.

Simulační část je místo, kde můj projekt pokulhává. Simulace ve svém aktuálním stavu postrádá řadu funkcí. Rozhodl jsem se místo vytváření mnoha pokročilých funkcí vytvořit solidní základ, na kterém je možné pokročilejší simulaci vystavět.

Systém základu simulace a kód pro kolize pochází ode mě za pomoci zdrojů o 2D analytické geometrii. Toto bylo největší úskalí projektu a rozhodnutí tvořit vlastní systém simulace byla chyba, která stála tolik úsilí, které chybělo na části projektu, které jsou aspoň vidět a na které jsem se při plánování těšil.

Dále po maturitě bych tedy rád pokračoval v přidávání dalších a dalších funkcí tomuto projektu a jako první určitě chci vylepšit způsoby přidávání prvků na obrazovku, jakožto i možnosti editoru prvků. Kromě toho se chci ve větším klidu podívat na to, jak se počítají interakce s rozličnými optickými prvky a přidat konečně Snellův zákon, který byl součástí mého původního plánu a který jsem musel kvůli nedostatku času na naprogramování systému na výpočty dalšího optického zákona prozatím opustit.

Během práce jsem se taky naučil s verzovacím systémem Git, který v repertoáru správného programátora nesmí chybět a který odteď budu pro své projekty používat. V rámci maturitního projektu jsem tedy získal i tuto nepostradatelnou schopnost.

Naučil jsem se i další, snad ještě víc důležitější věc, a tou je fakt, že rozvrhnout si projekt a posoudit, co vlastně zvládnou a co ne, je těžké. Při vymýšlení tématu jsem nepředvídal problémy, které pak při práci nastaly, a doplatil jsem na to.

Cíl, který jsem si stanovil na začátku byl rozsáhlý a až příliš ambiciózní a v průběhu tvorby jsem musel udělat kompromisy, které znamenaly, že jsem prioritizoval funkční a stabilní základ jakožto i GUI a neimplementoval jsem pokročilejší funkce v simulační části, jako třeba systém materiálů a Snellův zákon. I přesto se práce na simulaci stala nejvíce problematickou částí celé mé práce, kupříkladu na tvorbě kódu pro kolizi přímkou s úsečkou a následném řešení problémů s touto funkcí jsem se zasekl na několik týdnů a vlastně mi to zkazilo celý dojem z práce na projektu.

Nakonec se mi i tuto část ale podařilo doladit a provést přidání aspoň nejdůležitějších funkcí, které můj projekt potřeboval. Přestože v simulační části můj projekt postrádá hloubku, jsem s výsledkem svého snažení spokojený.